



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Retries and Dead Letters

The senior engineer answer sheet

One malformed record. Retried forever. The whole partition stopped behind it.

A retry is a prediction that something relevant may be different on the next attempt. If nothing can change, retrying is not resilience. It is repetition.

Almost every bad retry design in production is that one sentence being violated: retrying a failure that cannot succeed, or retrying in a place that costs more than the failure did.

This sheet is the written form of the episode. Kafka behaviour is sourced to Apache Kafka 4.3 documentation. Reasoning that is reasoning says so.

In scope

- Classifying a failure before choosing a response
- Why retrying in the handler costs you the consumer
- Retry topics, and the ordering they trade away
- Backoff, and what it is actually protecting
- The dead letter topic, and what belongs in it
- Replaying a dead letter safely
- What the failure path needs monitored

Out of scope

- Specific framework defaults, which are version-specific and rot fast
- A recommended retry count, which does not exist
- Producer level retries, which are a different mechanism
- Transactions, which are episode 7

Most retry questions are really classification questions wearing a retry costume.

01 Worked assumptions

Records per poll	500, the <code>max.poll.records</code> default
Blocking retries inside the handler	3
Delay between them	2 s, 4 s, 8 s
Time added to one poll by one bad record	14 seconds
<code>max.poll.interval.ms</code> default	300000, so 5 minutes
Bad records needed to breach it	about 21 in one batch

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 Classify first, retry second

Three questions, in this order. Can waiting change the outcome? Is the work safe to repeat? How late is the business willing for this to be?

A timeout, a connection refused, a 503: waiting can change those. A malformed payload, a validation failure, an account that does not exist: waiting changes nothing about them, ever.

A retry policy applied without this classification retries the unretryable, which is how a single bad record consumes a whole afternoon of capacity.

A retry is only useful when some relevant condition can differ on the next attempt; where nothing can differ, the attempt is repetition rather than resilience.

First principles, not vendor documentation

Retrying an operation that is not safe to repeat multiplies the damage of the failure rather than absorbing it.

First principles, not vendor documentation

04 Why retrying inside the handler costs you the consumer

Sleep and retry inside the listener and you are not just delaying that record. You are delaying the next call to poll.

Exceed `max.poll.interval.ms` and the group decides this consumer is no longer making acceptable progress, evicts it, and rebalances. That is the episode 3 failure, arriving from a different direction.

So a blocking retry is a decision to spend group membership on one record. Occasionally that is the right call. It is never a default.

`max.poll.interval.ms` is the maximum delay between invocations of poll when using consumer group management; if poll is not called before it expires, the consumer is considered failed and the group rebalances.

Kafka 4.3, Consumer Configs

Time spent retrying inside the handler is time not spent calling poll, so a blocking retry policy consumes the same budget that liveness detection uses.

First principles, not vendor documentation

05 The record that never succeeds

A permanently invalid record retried in place halts its partition. The consumer will not commit past an offset it refuses to complete, so nothing behind it moves either.

One malformed message, and every payment for every account that hashes to that partition is now waiting on it.

This is the case that makes a dead letter topic necessary rather than tidy. Something has to be allowed to move on.

A consumer cannot advance past an offset it will not commit, so a record that never succeeds blocks every later record in the same partition.

First principles, not vendor documentation

The blast radius of one poisoned record is its whole partition, not one key, because ordering within the partition is what is being preserved.

Kafka 4.3, Topics and Logs

06 Retry topics, and the ordering they trade away

Move the failure to a separate topic with a delay, and the main consumer keeps moving. The retry happens somewhere that costs nothing to block.

Now say the cost out loud, because this is the part interviews are actually probing. Event A fails and is delayed thirty seconds. Event B for the same account succeeds immediately. A comes back and succeeds SECOND.

Partition ordering is intact. BUSINESS ordering is inverted. That is a real trade, and the senior move is to make it knowingly and to say which accounts it is unacceptable for.

Retry topics preserve partition ordering while reordering the work, because the delayed record rejoins the flow after later records have already been processed.

First principles, not vendor documentation

A dead letter topic is an ordinary Kafka topic; nothing in the broker treats it specially, and the meaning is entirely a convention of the application.

Kafka 4.3, Topics and Logs

07 Backoff is protecting the dependency, not you

Immediate retry against an overloaded dependency adds load to the thing that is already failing. Every client doing it at once turns a slow dependency into a dead one.

Increasing delays give it room to recover. Jitter stops every client retrying on the same tick, which is the difference between a recovery and a synchronised second wave.

The exit condition is a business question, not a number from a blog post. How late may this payment be before late is worse than failed? That answer sets the ceiling.

Immediate retry against an overloaded dependency increases load on it, which is the standard argument for backoff.

First principles, not vendor documentation

There is no universally correct retry count; the exit condition follows from how late the business will tolerate the outcome being.

Operational practice, not vendor documentation

08 What belongs in the dead letter record

The original payload, unmodified. The original key, so a replay lands in the same partition. The original headers. The failure, with its class and its message. The number of attempts. When it was given up on.

A dead letter topic full of payloads with no failure context is a graveyard. Somebody will open it at three in the morning and learn nothing.

And a replay is only safe if the identity outlived the record: is the idempotency claim still there, or has it been cleaned up? Replaying without that check reprocesses work that already happened.

Preserving the original key on the dead letter record is what allows a replay to land in the same partition and keep per-key ordering meaningful.

Kafka 4.3, Topics and Logs

A replay is safe only where the idempotency identity for that operation still exists; if claims have been cleaned up, the replay is a new first attempt.

First principles, not vendor documentation

09 The failure path needs its own signals

Counting retries together with normal traffic hides both. The failure path is a traffic class of its own and it needs watching as one.

How many records are retrying now. Which failure type dominates. How old the oldest retry is. How fast the dead letter topic is growing.

Framework support can make retry infrastructure look almost trivial to configure, and that is exactly why the signals get forgotten. A retry system with no signals fails silently, which is the worst way for it to fail.

Retry and dead letter traffic measured together with normal traffic obscures both, because one is a small fraction of the other.

`Operational practice, not vendor documentation`

Configuring retry infrastructure through a framework reduces the effort required, which shifts the remaining risk onto whether anyone is watching it.

`Operational practice, not vendor documentation`

10 The interview answer

Classify the failure first. Retryable means waiting can change the outcome, and the work is safe to repeat.

Do not retry in the handler beyond a very short bound, because that spends the poll interval and gets the consumer evicted.

Use retry topics with backoff, and say the cost: partition ordering survives, business ordering can invert.

Dead letter what cannot succeed, with enough context to replay, and monitor the failure path separately from the happy one.

Naming the ordering cost of retry topics unprompted is the strongest single signal in an answer on this topic, because it is the part most explanations omit.

`First principles, not vendor documentation`

Questions to ask before you answer

1. Can waiting change this outcome at all?
2. Is the work safe to repeat?
3. How late may this be before late is worse than failed?
4. Where is the retry happening, and what does that block?
5. Does the delayed record reordering the work matter for this domain?
6. What context does the dead letter record carry?
7. Is the idempotency claim still alive when a replay happens?
8. Who is watching the retry depth, and what alerts on it?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.